Report On

Smart Store – POS and Inventory Management System

Submitted in partial fulfillment of the requirements of the Course project
in Semester III of Second Year Artificial Intelligence and Data Science

by
Mukund Kumar (Roll No. 69)
Ayan Shaikh (Roll No. 73)
Aaron Famy (Roll No. 74)

Supervisor
Mrs. Asitha Nadar



University of Mumbai

Vidyavardhini's College of Engineering & Technology

Department of Artificial Intelligence and Data Science



(2026-27)

Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

CERTIFICATE

This is to certify that the project entitled "Smart Store – POS and Inventory Management System" is a bonafide work of Mukund Kumar (Roll No. 69), Ayan Shaikh (Roll No. 73), Aaron Famy (Roll No. 74)" submitted to the University of Mumbai in partial fulfillment of the requirement for the Course project in semester III of Second Year Artificial Intelligence and Data Science engineering.

Supervisor	**Head of Department**
Mrs.Asitha Nadar	Dr. Tatawadarshi P. Nagarhalli

1. Abstract :-

The SmartStore – POS and Inventory Management System is a web-based application developed to simplify retail store management. It helps manage products, inventory, billing, customer information, and sales records in a centralized system, reducing manual work and errors.

The system allows administrators to add, update, delete, and search products while monitoring stock levels through low-stock warnings. Its POS module enables product selection, quantity entry, GST calculation, and bill generation with Cash, Card, and UPI payment options.

A Business Analytics Dashboard provides information such as daily revenue, total bills, total products, and low-stock products. The system uses HTML, CSS, and JavaScript for the interface, Java for backend processing, and MySQL for database management.

Overall, SmartStore improves billing accuracy, inventory tracking, and access to business information while providing a simple and efficient retail management solution.

Table of Contents :-

Chapter No		Title	Page No.
1		**Problem Statement**	**1**
	1.1	Block Diagram	3
2		**Module Description**	**4**
	2.1	Brief Description of Software and hardware	5
	2.2	Code	6
	2.3	Results and Conclusion	
3		**References**	**7**

Problem statement :-

Managing the daily operations of a retail store manually can be time-consuming and may lead to errors in product management, stock tracking, billing, GST calculation, and sales record maintenance. Traditional methods often require maintaining records in registers or separate systems, making it difficult to quickly access accurate information about available stock and previous transactions.

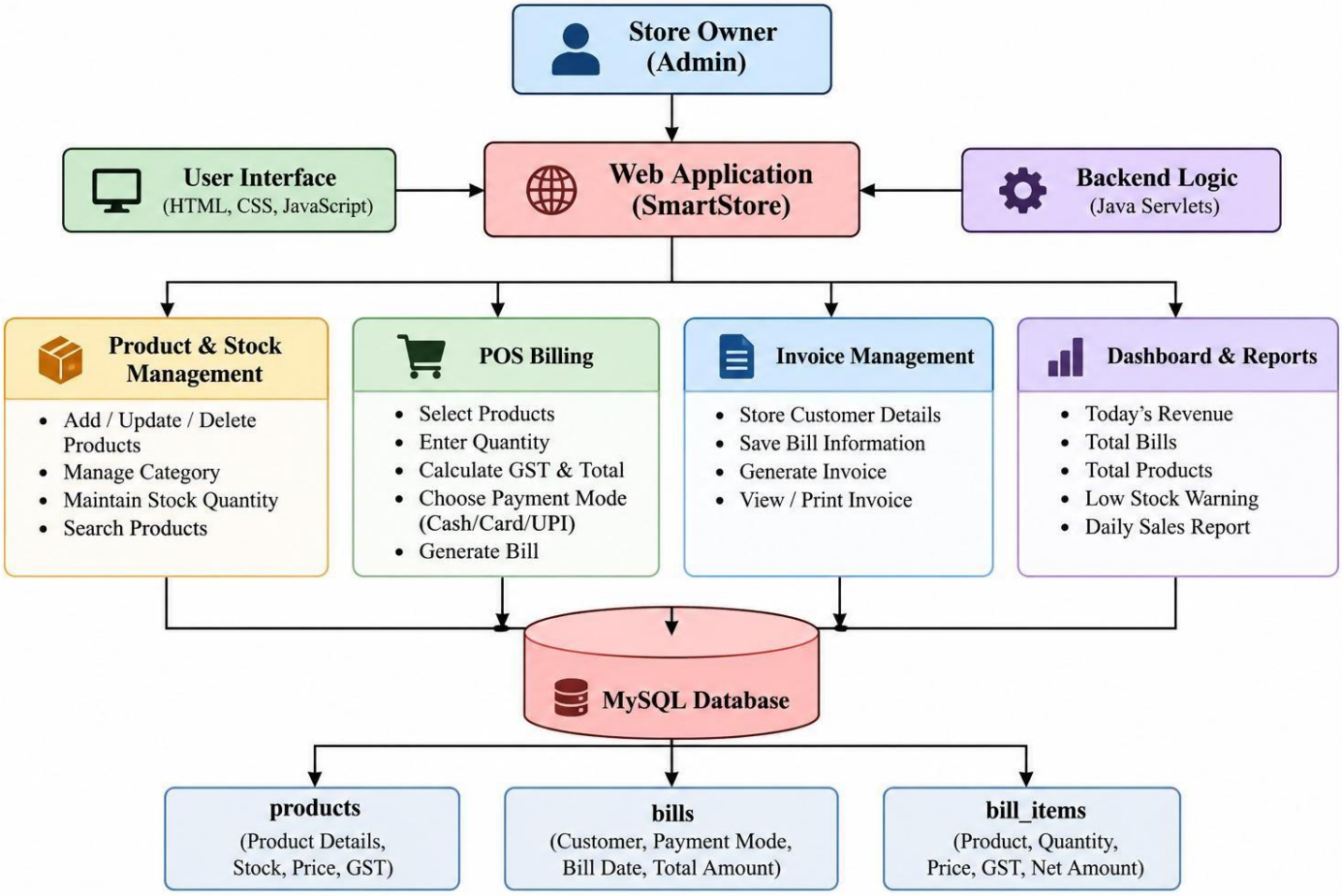
The main problem is to develop a simple and centralized system that can manage the complete retail store operations efficiently. The system should allow the store administrator to maintain product details, monitor stock quantities, create customer bills, calculate GST and total amounts, store transaction records, and generate invoices.

Therefore, the SmartStore – POS and Inventory Management System is developed to provide an integrated solution for inventory management, POS billing, invoice generation, and basic business analytics. The system aims to reduce manual work, minimize calculation errors, improve stock monitoring, and provide quick access to important store information.

Main Problems Addressed

- Manual maintenance of product and stock records.
- Difficulty in tracking available inventory.
- Errors in bill and GST calculations.
- Time-consuming manual billing process.
- Difficulty in maintaining customer and transaction records.
- Lack of quick information about sales and revenue.
- Difficulty identifying products with low stock.
- Manual preparation of invoices.

Block diagram :-



Module Description:-

The SmartStore – POS and Inventory Management System is divided into several modules. Each module performs a specific function and together they provide a complete solution for managing retail store operations. The main modules of the system are described below.

1. Product & Stock Management

This module is used to manage all product-related information and inventory. The administrator can add, update, delete, and search products. Each product contains details such as product name, category, barcode, price, GST, quantity, and unit. The module also helps the store owner monitor available stock and identify products with low stock.

2. POS Billing

The POS Billing module is used to create bills for customers. The user can select products, enter the required quantity, and add them to the billing cart. The system automatically calculates the subtotal, GST amount, and grand total. Different payment modes such as Cash, Card, and UPI can be selected. After completing the transaction, the bill is generated and stored in the database.

3. Invoice Management

This module manages the invoices generated after completing a sale. It stores information such as customer name, phone number, payment mode, bill date, purchased products, quantity, unit price, GST, and total amount. The generated invoice can be viewed and printed for the customer's reference.

4. Dashboard & Reports

The Dashboard module provides a quick overview of the store's business activities. It displays important information such as today's revenue, total number of bills, total products, low-stock products, and daily sales revenue. The dashboard helps the store owner monitor sales performance and inventory status easily.

5. Database Management

The database module is responsible for storing and managing the application's data using MySQL. Important tables such as products, bills, and bill_items are used to maintain product

information, billing records, and item-wise transaction details. This ensures that the data is stored systematically and can be retrieved whenever required.

6. User Interface

The User Interface provides a simple and user-friendly way to interact with the system. It contains navigation options for Dashboard, POS Billing, and Products & Stock. The interface is developed using HTML, CSS, and JavaScript and provides forms, tables, buttons, and other elements required to perform different operations.

Brief description of software & hardware used and its programming

Software Requirements :-

- **Operating System:** Windows 10 / Windows 11
- **Programming Language:** Java • **Frontend Technologies:** HTML, CSS, JavaScript
- **Backend Technology:** Java Servlets / JSP
- **Database:** MySQL
- **Database Tool:** MySQL Workbench
- **Web Server:** Apache Tomcat
- **IDE:** Eclipse / IntelliJ IDEA / VS Code
- **Browser:** Google Chrome / Microsoft Edge
- **Database Connectivity:** JDBC (Java Database Connectivity)

Hardware Requirements :-

The system can run on a standard computer or laptop with the following minimum hardware configuration:

- **Processor:** Intel Core i3 or equivalent
- **RAM:** Minimum 4 GB
- **Storage:** Minimum 10 GB free space
- **Display:** 1366 × 768 resolution or higher
- **Keyboard and Mouse:** Standard input devices
- **Network:** Local network/internet connection when required

Programming Used

The project primarily uses Java for backend programming. Java Servlets handle requests from the web interface and perform operations such as product management, billing, invoice generation, and database communication.

Code / Program :-

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>SmartStore POS &amp;amp; Inventory</title>
  <link rel="stylesheet" href="style.css">
</head>

<body>
<header class="navbar">
  <div class="logo">
    <div class="logo-box">▣</div>
    <div>
      <h2>SmartStore</h2>
      <small>POS &amp;amp; Inventory</small>
    </div>
  </div>
  <nav class="nav">
    <a href="Main" class="active">▥ Dashboard</a>
    <a href="Main?action=billing">🛒 POS Billing</a>
    <a href="Main?action=products">▦ Products &amp;amp; Stock</a>
  </nav>
</header>

<div class="container">
  <div class="heading">
    <h1>📈 Business Analytics Dashboard</h1>
    <p>Real-time overview of sales, transactions, and stock health</p>
  </div>
  <div class="cards">
    <div class="card stat">
      <div class="icon">₹</div>
      <div>
        <small>TODAY'S REVENUE</small>
        <h2>₹{{REVENUE}}</h2>
      </div>
    </div>
    <div class="card stat">
      <div class="icon">🛒</div>
      <div>
        <small>TOTAL BILLS</small>
        <h2>{{BILLS}}</h2>
      </div>
    </div>
  </div>
```

```
  <div class="grid">
      <div class="card">
        <div class="section-title">
          <h2>Daily Sales Revenue (Last 7 Days)</h2>
        </div>
        {{CHART}}
      </div>
      <div class="card">
        <div class="section-title">
          <h2>⚠ Low Stock Warnings</h2>
        </div>
        {{LOW_STOCK_LIST}}
      </div>
    </div>
    <div class="card table-card">
      <table>
        <tr>
          <th>Invoice</th>
          <th>Customer</th>
          <th>Payment</th>
          <th>Total</th>
          <th>Action</th>
        </tr>
        {{RECENT_BILLS}}
      </table>
    </div>
</div>
</body>
</html>
```

Main.java:-

```
import jakarta.servlet.ServletException; import
jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

import java.io.*; import
java.math.*; import
java.sql.*; import
java.time.LocalDate;
import java.time.format.DateTimeFormatter; import
java.util.*;

@WebServlet(urlPatterns = {"/Main", "/main"}) public
class Main extends HttpServlet {

   private static final String URL = env("DB_URL",
         "jdbc:mysql://localhost:3306/stock_billing?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC&
characterEncoding=UTF-8");
   private static final String USER = env("DB_USER", "root");
```

```
    private static final String PASSWORD = env("DB_PASSWORD", "viva123");

    private static final int LOW_STOCK = 5;
    private static final List<String> PAYMENTS = List.of("CASH", "CARD", "UPI");
 static {       try {
          Class.forName("com.mysql.cj.jdbc.Driver");

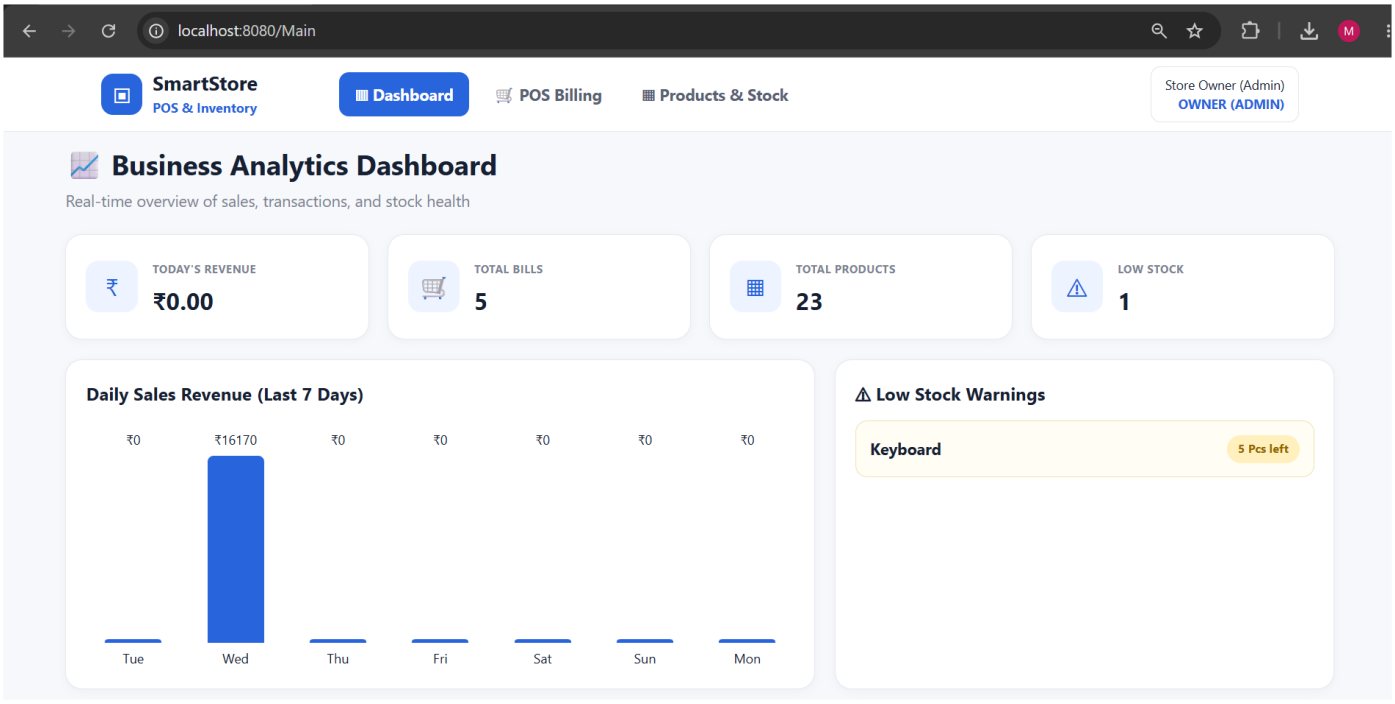
        } catch (ClassNotFoundException ignored) {
        }
    }
   private static String env(String key, String def) {
        String v = System.getenv(key);
return v == null ? def : v;
    }
    private static class Line {
int id, stock, qty;         String
name, unit;
        BigDecimal price, gstPct;
BigDecimal subtotal() {
           return price.multiply(BigDecimal.valueOf(qty));
        }
        BigDecimal gst() {
           return subtotal().multiply(gstPct)
                   .divide(BigDecimal.valueOf(100), 2, RoundingMode.HALF_UP);
        }
    }
    private Connection connect() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
    private String template() throws IOException {
InputStream in = null;         try {
           if (getServletContext() != null) {
              in = getServletContext().getResourceAsStream("/index.html");
if (in == null) {
                 in = getServletContext().getResourceAsStream("index.html");
              }
           }
        } catch (Exception ignored) {
        }
        if (in == null) {
           in = Main.class.getResourceAsStream("/index.html");
        }
        if (in == null) {
           in = Main.class.getClassLoader().getResourceAsStream("index.html");
        }
        if (in == null) {
           File f = new File("index.html");
if (f.exists() && f.isFile()) {
              in = new FileInputStream(f);
           }
}
```

```
        if (in != null) {
            try (InputStream input = in) {
                return new String(input.readAllBytes(), StandardCharsets.UTF_8);
            }
        }
private void invoicePage(HttpServletRequest req, HttpServletResponse resp) throws Exception {
 int id = parseInt(req, "id");
      if (id <= 0)          throw new
IllegalArgumentException("Invalid invoice ID");

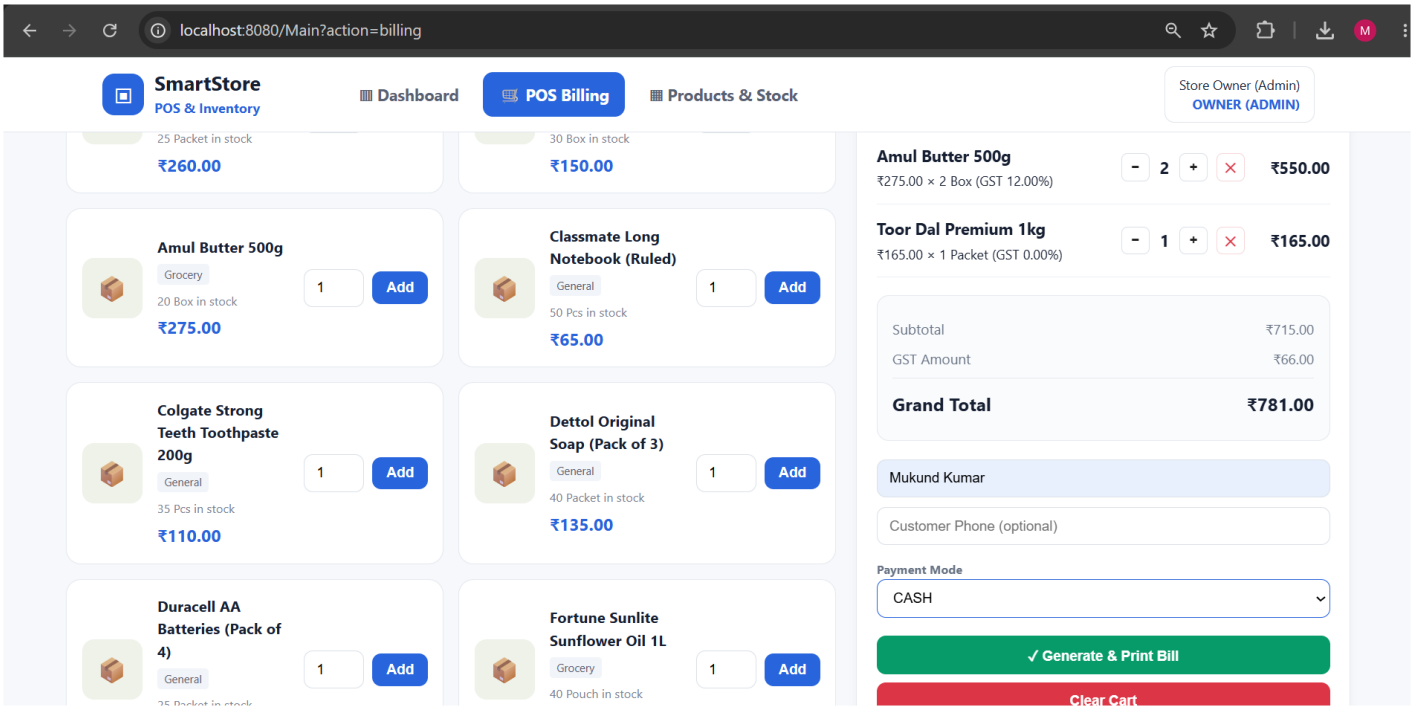
        String customer = "", phone = "", payment = "", date = "";
        StringBuilder rows = new StringBuilder();
    BigDecimal sub = BigDecimal.ZERO;       BigDecimal
    gst = BigDecimal.ZERO;       try (Connection con =
    connect()) {
          try (PreparedStatement ps = con.prepareStatement(
                "SELECT customer,phone,payment_mode,bill_date FROM bills WHERE id=?")) {
ps.setInt(1, id);
              try (ResultSet rs = ps.executeQuery()) {
if (!rs.next())
                    throw new IllegalArgumentException("Invoice #" + id + " not found.");
customer = rs.getString("customer");                    phone = rs.getString("phone");
                payment = rs.getString("payment_mode");
Timestamp ts = rs.getTimestamp("bill_date");                    date
= ts != null
                        ? ts.toLocalDateTime().format(DateTimeFormatter.ofPattern("dd MMM yyyy, hh:mm a",
Locale.ENGLISH))
                        : "N/A";
              }
          }
          try (PreparedStatement ps = con.prepareStatement(
                "SELECT COALESCE(p.name, 'Item #' || bi.product_id) AS item_name, "
                      + "bi.quantity, bi.price, bi.gst_percent, bi.total, bi.gst_amount "
                      + "FROM bill_items bi LEFT JOIN products p ON p.id=bi.product_id "
                      + "WHERE bi.bill_id=? ORDER BY bi.id ASC")) {
ps.setInt(1, id);
              try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    BigDecimal itemTotal = rs.getBigDecimal("total");
BigDecimal itemGst = rs.getBigDecimal("gst_amount");
sub = sub.add(itemTotal);                       gst = gst.add(itemGst);
                    rows.append("<tr><td>").append(esc(rs.getString("item_name")))
                          .append("</td><td>").append(rs.getInt("quantity"))
                          .append("</td><td>₹").append(money(rs.getBigDecimal("price")))
                          .append("</td><td>").append(money(rs.getBigDecimal("gst_percent")))
                          .append("% (₹").append(money(itemGst)).append(")")
                          .append("</td><td>₹").append(money(itemTotal.add(itemGst)))
.append("</td></tr>");                    }
              }
          }
        }
        String body = "<div class='invoice'>"
```

```
            + "<div class='invoice-head'><div><h2>SmartStore</h2><small>POS &amp;amp; Inventory
Management</small></div>"
            + "<div style='text-align:right'><h2>Invoice #" + id + "</h2><small>" + esc(date) + "</small></div></div>"
+ "<div class='invoice-meta'>"
            + "<p><b>Customer Name:</b> " + esc(customer) + "<br>"
            + "<div class='line grand'><span>Grand Total (Paid)</span><span>₹" + money(sub.add(gst)) + "</span></div>"
            + "</div>"
            + "<a class='btn light' href='Main?action=billing'>+ New Bill</a> "
            + "<a class='btn light' href='Main'>Dashboard</a></div>"
            + "</div>";
      resp.getWriter().println(page("", "Invoice #" + id, body));
}
}
```

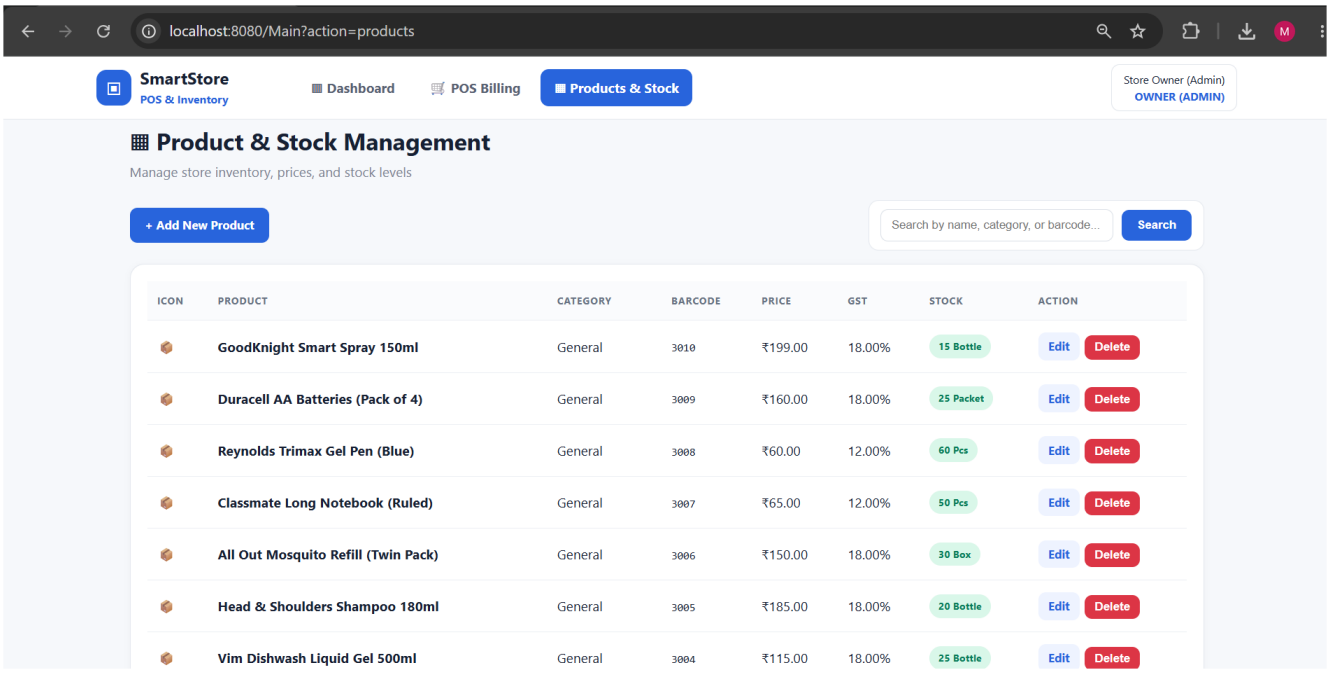
Code Output :-



Dashboard View



POS Billing View



Product & Stock Management View

2.3 Results and conclusion

The SmartStore – POS and Inventory Management System successfully provides an efficient and user-friendly solution for managing retail store operations. The system integrates product management, stock management, POS billing, invoice generation, and business analytics into a single platform.

The system reduces manual work and minimizes errors in billing, GST calculation, and inventory management. It also helps the store administrator maintain accurate product and transaction records and quickly identify low-stock products. The dashboard provides useful information about revenue, bills, products, and sales, which helps in monitoring the overall performance of the store.

The use of Java, HTML, CSS, JavaScript, and MySQL makes the system reliable, organized, and easy to maintain. Overall, the project achieves its main objective of providing a simple, accurate, and effective digital solution for retail store management

References :-

1. Shopify. (2026). *Shopify POS – Retail Point of Sale and Inventory Management*. Shopify. Website: https://www.shopify.com/in/pos/retail-pos

2. Square. (2026). *Square for Retail – Point of Sale and Inventory Management*. Squar Website**:** https://squareup.com/us/en/retail

3. Clover. (2026). *Clover Retail POS – Point of Sale and Inventory Management*. Clover. Website**:** https://www.clover.com/pos-solutions/retail

4. Toast. (2026). *Toast Retail POS and Inventory Management*. Toast. Website**:** https://pos.toasttab.com/products/retail-inventory-management

5. Loyverse. (2026). *Loyverse POS – Point of Sale and Inventory Management*. Website:https://loyverse.com/en-us/advanced-inventory